

Rev80 — Project Progress

Maps client meetings, stated requirements, and development phases from `VibeGui Project.md` to git commits. Useful for tracking what has been delivered and what remains.

Project Background

Originated 2025-01-23 when a Digiducer-based data collection script was demonstrated to Carmen at a client site. The goal: replace expensive commercial vibration analysis tools (e.g. VibeAnalyze, \$999) with a purpose-built Python desktop app targeting industrial predictive maintenance workflows.

Development Phases

Proto — Initial prototype (Feb – Aug 2025)

Bare-bones data capture and display using the Digiducer USB audio interface. No HDF5, no velocity spectrum, minimal GUI.

Date	Commit	Description
2025-02-15	<code>ae1736e</code>	Project created
2025-03-05	<code>5b017d1</code>	Project management notes
2025-03-17	<code>891d26f</code>	Backup snapshot
2025-05-20	<code>cca1d4e</code>	Working demo (Digiducer stream + basic plot)
2025-05-20	<code>e247dba</code>	Module init
2025-05-21	<code>856f855</code>	Digiducer abstracted into sensor and logger types
2025-08-12	<code>1257994</code>	First working dearpygui GUI with live stream
2025-08-12	<code>a3c2fd1</code>	Project management
2025-08-12	<code>be9e91c</code>	Bug fix: array dimensions

Meeting — 2025-01-23: Carmen, first demo

Requirements captured:

#	Requirement	Status
R1	Frequency range and bin size selector	✖ Done — Dec 2025
R2	Normalize FFT to IPS or g (velocity preferred over acceleration)	✖ Done — Dec 2025
R3	Display top peaks	Done — Dec 2025
R4	Smoothing	Not implemented
R5	Interactive peak inspection	Not implemented
R6	Running speed harmonics markers	Not implemented

#	Requirement	Status
R7	Bearing fault frequency markers	Not implemented
R8	Metadata: date, company, machine info, bearing types, running rate	Not implemented
R9	Trend: overall vibration over time	Done — Mar 2026
R10	Luxury: startup and coastdown trend	Future

Phase 0 — Digiducer acquisition, GUI polish (Nov – Dec 2025)

Core pipeline stabilised; units, HDF5 persistence, and modular architecture added.

Date	Commit	Description
2025-11-17	587e393	Save button added
2025-11-20	e9120d2	Logging module with YAML config; print statements removed
2025-11-25	c8bb0ae	VibeSample refactored
2025-11-26	6049f7f	Carmen meeting notes — deployment roadmap
2025-12-11	22014af	Units switching; save/load via pickle
2025-12-16	10e48a0	scipy FFT tools; GUI unit and integration controls
2025-12-17	580d254	Logger made universal
2025-12-17	383f28e	File browser fix
2025-12-17	14dd5ca	Unit conversion consistency
2025-12-17–18	772214e – 4424d3d	AcquisitionSettings upgrade (multi-step)
2025-12-29	38f22d3	Plot units upgrade — units switching throughout GUI
2025-12-29	8fb488c	refactor_util — split into sample, sensor, simulation, digiducer modules

Meeting — 2025-11-25: Carmen, deployment roadmap

Phase definitions agreed:

Phase	Scope	Budget
Phase 1	Development to date (Digiducer era)	\$2k
Phase 2	Polish and deploy to Windows	\$1k
Phase 2b	Integrate BNC DAQ for proximity probes	\$1k
Phase 3	Sales	—

Requirements from this meeting:

#	Requirement	Status
R11	Switch between velocity and acceleration in frequency domain	Done — Dec 2025 (10e48a0 , 38f22d3)

#	Requirement	Status
R12	IPS zero-to-peak trend analysis	Done — Jan 2026, extended Mar 2026
R13	RMS amplitude option	Done — Mar 2026 (5872cab , 1e040d3)
R14	Two simultaneous sensors (phasing)	Done — Mar 2026 (multi-channel pipeline)
R15	Proximity probe / DAQ support	Partially done — PicoScope replaces BNC DAQ

Meeting — 2025-12-30: Carmen

Requirements from this meeting:

#	Requirement	Status
R16	Overall vibration energy (IPS 0-P)	Done — Jan 2026 (ff4b31d)
R17	Trend plot of overall vibration over time	Done — Mar 2026 (cff702e)
R18	Fix IPS unit scaling (was ~200× wrong)	Done — Dec 2025 / confirmed Jan 2026 (e7eddde)

Phase 1b — HDF5, FFT validation, overall vibration (Jan 2026)

Date	Commit	Description
2026-01-05	dc64e37	UI_Elements moved to util for global access
2026-01-05	e3a988d	AcquisitionSettings hot-swap while streaming
2026-01-05	cde0834	Reorg and cleanup
2026-01-06	32ab828	HDF5 save/load (replaces pickle)
2026-01-09	e76fb4f	DAQ hardware evaluation notes committed
2026-01-10	f0c549e	ISO timestamp; monotonic rel_time per block
2026-01-11	e7eddde	FFT amplitude scaling validated with single-tone tests
2026-01-12	ff4b31d	Overall 0-P vibration display (acceleration and velocity)
2026-01-12	fcf9e7e	Oversample stream for better Welch FFT resolution
2026-01-16	70c51db	uldaq dependency added (MCC USB-1608FS-Plus evaluation)
2026-01-20	2f95891	MCC DAQ code examples
2026-03-09	2508756	MCC DAQ integration work

Meeting — 2026-01-11

Notes: - Overall 0-P vibration display added and demonstrated - Bin selector behaves poorly — flagged - Timestamp now FAT32-compatible (colons replaced by hyphens) - UI/UX language and jargon should match Alta's tooling - Demo screenshots needed for website

#	Requirement	Status
R19	Confirm overall vibration values are correct (vs. individual peaks)	[*] Done — validated <code>e7eddde</code>
R20	Fix bin selector behaviour	Done — Mar 2026 (<code>a69f14f</code> , <code>3fde4bc</code>)
R21	UI/UX language aligned with Alta	Ongoing
R22	Demo screenshots for website	Not committed

Phase 2 — PicoScope 4000A acquisition backend (Mar 2026)

Hardware pivot: replace Digiducer/sounddevice with PicoScope 4000A. Enables any IEPE sensor (accelerometers, proximity probes, microphones) via standard BNC input — removes Digiducer hardware dependency.

Date	Commit	Description
2026-03-13	<code>cf89848</code>	PicoScope 4000A acquisition backend — <code>FindPicoScope()</code> , <code>PicoScopeStream</code> , ADC → mV, accumulator, USB power fallback
2026-03-13	<code>1b711c9</code>	Merge feature/picoscope into main
2026-03-13	<code>2b9aa38</code> <code>d9add87</code>	Comprehensive README added
2026-03-13	<code>91b06ce</code>	CLAUDE.md architecture guide added
2026-03-16	<code>8abb72d</code>	IEPE sensor registry, triax support, PicoScope GUI integration — <code>ScopeSensor</code> dataclass, <code>ScopeSensorRegistry</code> YAML persistence, per-channel sensitivity (mV/EU), GUI sensor library panel
2026-03-16	<code>cd40f63</code>	Harden PicoScope detection: overvoltage recovery, watchdog (5 s timeout, 3 reconnect attempts)
2026-03-16	<code>f03255d</code>	Fix handle-close between retries; extended delay after <code>PICO_NOT_RESPONDING</code>
2026-03-16	<code>67ebfd6</code>	Test: update for new default voltage range

Phase 2a — Multi-channel EU pipeline and GUI overhaul (Mar 2026)

Full multi-channel data path from hardware to display, with per-channel sensor assignment, modality-aware integration, and three-column GUI layout.

Date	Commit	Description
2026-03-18	<code>0d8911c</code>	Three-column GUI layout with primary window resizing
2026-03-18	<code>0147231</code>	Modality-aware integration (acc↔vel↔disp via FFT); mV/ EU sensitivity pipeline; unit conversion tests

Date	Commit	Description
2026-03-19	13b1095	Add mil (1/1000 in) as displacement unit
2026-03-20	afa5ce0	Fix frequency-domain integration using signed exponent
2026-03-24	09cea74	Multi-channel pipeline, scope sensor registry, EU unit refactor — PicoScopeStream multi-channel buffers; per-channel mV → EU via ScopeSensor; DataCollector fan-out
2026-03-24	4f8897c	GUI overhaul — channel config panel; per-channel status lights; dual-axis plots; scope sensor registry UI
2026-03-24	8280bd5	Test restructure: multi-channel, hardware isolation, new API coverage
2026-03-24	5d57ab4	Fix PicoScope circular buffer wraparound in streaming callback
2026-03-24	0ca1251	Replace Acquire/Configure tabs with streamlined left-panel layout
2026-03-24	c70b8f3	Visual section containers (borders, themed backgrounds) for left panel
2026-03-25	cff702e	Multi-channel data pipeline — 32-frame ring cache; ChannelResult frozen dataclass; trend store; multi-channel HDF5 layout; browse ← → buttons
2026-03-25	142dc30	Expanded multi-channel test coverage; TestFrameCache, TestTrend, TestChannelResult
2026-03-25	32410b6	Fix: overflow bitmask; default voltage range 10 → 7 (10× finer ADC resolution)
2026-03-25	23d0902	Test: overflow key handling, per-channel helpers
2026-03-25	e365099	Per-channel AC/DC coupling; results card sizing; 3 default peaks
2026-03-25	5ec3e45	Fix: reset channel config and plot series on device switch

Phase 2b — Signal generator, spectrum controls, GUI polish (Mar 2026)

AWG output for excitation testing; FFT window selection; manual axis control; left-panel redesign; siggen config persistence.

Date	Commit	Description
2026-03-25	f74dd41	Signal generator UI — Generate config tab (enable, waveform, freq, amplitude, offset); SimulatedSensor buried (CI/test only via VibeSensor.simulated())
2026-03-25	2a05ff9	FFT window type selector (Hann/Blackman-Harris/Flattop/Hamming/Boxcar/Bartlett)
2026-03-25	7e6b89f	Manual axis control — remove auto-fit on redraw; Autoscale button; Clear Cache button; full Welch frequency range (remove maxfreq display crop)
2026-03-25	ffb4121	Left panel rework — Device card, Channels card, button visibility fix (width=-1 bug), Load button restored

Date	Commit	Description
2026-03-25	83a980d	Card heights (DPG autosize_y fix); toggle button colour theming (idle/waiting/active); autoscale on first acquisition
2026-03-25	ef0b560	Channel Setup and Signal Generator buttons in Channels card
2026-03-25	d0c1068	Device card expanded with model, S/N, ID, channel count
2026-03-25	7bc729d	CH_WARNINGS_SECTION height scales to overflow channel count
2026-03-25	eb811b6	Siggen config persisted to <code>channel_assignments.yaml</code> ; restored on device connect
2026-03-25	7aa69b3	Unified VibeSample pipeline — <code>process()</code> replaces <code>get_accel()</code> + <code>fft()</code> ; returns <code>ChannelResult</code> ; cross-modality FFT integration for physically correct time-domain signal
2026-03-25	5872cab	Amplitude mode selector (RMS / 0-P / P-P) in GUI; Y-axis labels per mode
2026-03-25	1e040d3	Amplitude mode moved to <code>ScopeSensor</code> (per-sensor, not global spectrum setting)
2026-03-26	a69f14f	Modernised AcquisitionSettings — derived properties (samplerate, blocksize from maxfreq/binsize); explicit highpass/lowpass filter fields; Welch overlap; <code>n_fft_bins</code> ; <code>memory_bytes</code>
2026-03-26	ad98816	Lowpass filter added to collector pipeline
2026-03-26	3fde4bc	Spectrum tab redesign — combo presets; live-updating derived fields; filter controls
2026-03-26	36db391	Tests updated for new AcquisitionSettings API

Meeting — 2026-03-18

#	Requirement	Status
R23	Multi-channel: toggle channels on/off	Done (09cea74 , 4f8897c)
R24	Two-channel phase relationship display	Not implemented
R25	Time-domain zoom window (e.g. 5 ms)	Not implemented
R26	Track down velocity integration low-frequency noise	Partially addressed (afa5ce0); warrants further investigation
R27	Windows packaging (exe)	Not implemented

Meeting — 2026-03-25

#	Requirement	Status
R28	0-P, P-P, RMS selectable per sensor	Done (5872cab , 1e040d3)

#	Requirement	Status
R29	Acquisition settings causing long capture times	Done — AcquisitionSettings now derives samplerate / blocksize from maxfreq / binsize with correct power-of-2 arithmetic (a69f14f)
R30	Displaying wrong bins in spectrum setup	Done — Spectrum tab redesigned with live derived fields (3fde4bc)

Phase 4 — mV-domain pipeline + trend unit reflow (May 2026)

VibeSample refactored to store raw mV throughout; sensitivity conversion and Butterworth filtering moved to DataCollector.process_sample. Welch PSD cached per sample (keyed to filter/welch config); five broadband RMS overalls (integration orders -2...+2, in mV RMS) cached per sample and stored per trend timestep as a (M,5) ndarray. Unit/sensitivity/amplitude-mode changes reflow the trend at display time via DataCollector.get_trend_for_display() — no clearing required. DataCollector.process_samples() introduced as the single entry point for frame processing. GUI display loop simplified. HDF5 bumped to v4 with per-channel orders matrix; v3 files loaded in best-effort degraded mode.

Date	Commit	Description
2026-05-05	0c76c9a	refactor: mV-domain VibeSample + 5-order trend reflow
2026-05-05	5fcf09a	refactor(gui): simplify display loop via process_samples()
2026-05-05	3717286	test: update for mV-domain pipeline
2026-05-05	0a1b38c	feat(gui): configurable frame cache depth with recording-window display
2026-05-27	90a97c6	feat(picoscope): enumerate all connected scopes; hardware channel count probe

Requirements added in Phase 4

#	Source	Requirement
R28	May 2026	Configurable frame cache depth (integer selector in Acquisition config tab) with live recording-window and total-memory derived displays

Phase 5 — GUI responsiveness profiling (Sep 2026)

experimental/profiling . The GUI stuttered on every processing call, worse with every enabled channel, after the mandatory anti-alias filter and oversampled streaming landed. Built per-stage timing (src/rev80/_profile.py , rev80 --profile , scripts/profile-pipeline) before changing anything, then fixed the three causes it indicted: an unbounded raw->display resample ratio designing a 512821-tap FIR per channel per frame on hardware only, the vendor adc2mV per-sample Python loop holding the GIL inside the driver callback, and a per-channel peaks table destroyed and rebuilt every frame. See doc/CHANGELOG.md for the measured before/after on each.

The standing tool. scripts/profile-pipeline sweeps channel counts and prints a per-stage table, simulated or against real hardware. Its --raw-rate defaults to the hardware-realistic clock rather than the nominal one,

because the largest defect found here was invisible at exactly 25600 Hz – the whole test suite passed at 0.64 ms per call while the instrument stalled at 73.6. A performance claim measured only against `SimulatedSensor` is not a measurement.

Requirements added in Phase 5

#	Source	Requirement
R47	Sep 2026	Settling indicator while the measurement chain stabilises

Requirements Tracker

#	Source	Requirement	Status
R4	Jan 2025	Spectral smoothing	Abandoned
R5	Jan 2025	Interactive peak inspection (click peak → identify frequency)	Done — crosshairs on spectrum, time-series, and trend plots (<code>7f9b6a8</code>)
R6	Jan 2025	Running speed harmonic markers on spectrum — tachometer channel provides live 1xRPM (feature/tachometer); harmonic overlay implementation pending	TODO
R7	Jan 2025	Bearing fault frequency markers on spectrum	Abandoned — future capability, deferred for simplicity
R8	Jan 2025	Measurement metadata (company, machine, bearing types, running rate)	Done — measurement notes field (<code>2623d24</code>)
R10	Jan 2025	Startup / coastdown trend (luxury)	Done — trend plot with browse (<code>cff702e</code> , <code>142dc30</code>)
R21	Jan 2026	UI/UX language aligned with Alta tooling	Done — plot labels, dynamic legend, Y-axis units (<code>7f9b6a8</code>)
R22	Jan 2026	Demo screenshots for website	TODO
R24	Mar 2026	Multi-channel advanced plots: phase relationship, orbit plot (XY scope), waterfall, polar plot — many enabled by tachometer keyphasor channel	TODO
R25	Mar 2026	Time-domain zoom window	Done — time-series autoscaled to 300 ms window (<code>7f9b6a8</code>)
R26	Mar 2026	Velocity integration low-frequency noise investigation	Done — FFT integration sign fix + Welch window leakage fix (<code>afa5ce0</code> , <code>3e4bb59</code>)
R27	Mar 2026	Windows packaging (exe)	Done — Rev80Setup installer (<code>694146d</code> , <code>rev80.iss</code> , <code>build.sh</code>)

#	Source	Requirement	Status
R31	Aug 2026	Rebrand vibechecker → Rev80	Done (62dd2c2)
R32	Aug 2026	Anti-aliasing protection — field incident: high-frequency bearing fault aliased at low power, injecting spurious spectral energy. Fixed via mandatory oversample → zero-phase digital anti-alias filter → decimate in <code>PicoScopeStream</code>, plus a widened Nyquist margin (<code>samplerate = nextpow2(2.56*maxfreq)</code>, the 2.56x convention real FFT vibration analyzers use) so the filter has real transition-band room; retired the old user-facing (off-by-default, ineffective) lowpass control. Related fixes from the same hardware investigation: a new streaming-rate watchdog flags sustained USB throughput degradation that the existing silence watchdog couldn't see, and <code>MAXFREQ_PRESETS</code> ' top three entries (100k/250k/500k Hz) were dropped — they requested sample rates far beyond this hardware's measured continuous-streaming ceiling and were already silently corrupting most captured data.	Done — see working tree (not yet committed)
R33	Aug 2026	Flexible plot layout / dockable panels — users need context-dependent layouts (e.g. spectrum + waveform vs. trend + spectrum). Investigate DPG drag-drop window docking; if not natively supported, evaluate panel-switching or split-view alternatives.	TODO
R34	Aug 2026	ISO 20816-3 harmonization — evaluation of machine vibration measured on non-rotating parts, for industrial machines >15 kW at 120–15000 rpm: the class covering the blowers and motors this tool targets. Needs velocity RMS over a <i>declared</i> 10–1000 Hz band (2–1000 Hz below 600 rpm), machine-class selection, and Zone A/B/C/D boundaries with zone-based alarming. Current gaps: the overall is not band-limited — it spans <code>highpass_fc ... fs/2</code>, up to $2.05 \times F_{\text{max}}$, measured at +25% error from content the user explicitly excluded via <code>F_max</code> — and there is no machine classification or zone display, so the app reports a velocity number the user cannot interpret without an external table.	TODO

#	Source	Requirement	Status
R35	Aug 2026	ISO 2954 instrument conformance — requirements for instruments measuring vibration severity. Needs $\pm 10\%$ amplitude accuracy over 10–1000 Hz, a measurement band that is declared and stored with the data, frequency response within tolerance at the band edges, and overload indication. Current gaps: the band is undeclared and not persisted; the 4th-order highpass at 10 Hz puts -3 dB exactly on the lower band edge rather than in the passband (corner should sit at ~ 2 –5 Hz for a 10 Hz edge, or the response be compensated). Overload detection/exclusion is already done.	TODO
R36	Aug 2026	ISO 13373-1/-2 conformance — condition monitoring: measurement procedures (-1) and processing/presentation of vibration data (-2). Substantially met already: the v4/v5 HDF5 stores the full acquisition, channel and sensor snapshot alongside the data. Remaining: a machine / measurement-point hierarchy and route concept (currently only free-text channel names and notes), and preventing or flagging a trend assembled from frames acquired under different $F_{\max}$ or band settings, which is not comparable.	TODO
R37	Aug 2026	ISO 5348 mounting guidance — mechanical mounting of accelerometers. Documentation rather than code: usable frequency range is dominated by the mount (stud $\gg$ adhesive $\gg$ magnet $\gg$ handheld probe), and a handheld probe is unusable much above ~ 1 kHz. The app will happily display a 10 kHz spectrum the mount cannot physically support, with no indication. Add mounting guidance to the README, and ideally a per-measurement mount-type field driving a usable-bandwidth warning on the spectrum.	TODO
R38	Aug 2026	Calibration traceability (ISO 16063) — ScopeSensor carries a sensitivity typed in from the sensor datasheet, with no calibration date, method, or reference standard, so a saved measurement cannot be tied to a traceable calibration. Add calibration metadata to the sensor record and persist it with each measurement. A calibrated shaker check (0.1 in/s @ 191 Hz, 100 mV/g) put the end-to-end chain within $\sim 1.6\%$, so the accuracy is there — the provenance is not.	TODO

#	Source	Requirement	Status
R41	Aug 2026	Envelope / demodulation analysis — for rolling-element bearings this is <i>the</i> diagnostic: a defect's impulses ring a housing resonance at 2–20 kHz and are buried under the 1x in the raw spectrum, but appear as a clean line at the defect rate with $\pm 1x$ load-zone sidebands in the envelope, months earlier. Implemented in <code>rev80/envelope.py</code> with an Envelope plot tab and automatic demodulation-band selection. Verified against the simulated oracle at 125x SNR. Remaining: trending and alarming on the envelope line, which needs a defect-rate input (bearing geometry or a tachometer) and so depends on R6/R24; and drawing the demodulation band edges on the spectrum plot.	Done (band trending pending R6/R24)
R42	Aug 2026	Crest factor and kurtosis — two impulsiveness scalars a broadband overall averages away entirely. Crest factor rises early in a bearing defect's life and falls once it spalls; kurtosis above ~4 flags repetitive impacts. On the result card, trended and persisted. Remaining: no trend <i>plot</i> (the trend plot's two y-axes are unit-based and these are dimensionless — needs a third axis, an R33 layout decision) and no alarming (<code>FixedThresholdHook</code> is unit-aware and would need a dimensionless mode — belongs with R39).	Done (plot/alarm pending R33/R39)
R39	Aug 2026	Spectral anomaly hook — fix or remove. Unwired from the GUI panel (<code>GUI_ANOMALY_HOOK_TYPES</code>); the hook and the headless front end are untouched. It triggers on <code>np.any(abs(spec - baseline)/baseline > threshold)</code> across every bin, but Welch runs a single segment in every shipped preset (<code>nperseg == blocksize</code>), so each noise-floor bin is chi-squared(2) with a standard deviation equal to its own mean. P(some bin of ~2000 exceeds 1.5x) is ~1.0 on healthy data, which makes <code>consecutive_n</code> a delay rather than a defence; observed firing at the earliest frame it arithmetically can. Bins 0 and 1 are additionally hard-zeroed for integration, so on any velocity/displacement channel they deviate by ~1e12 and fire permanently. A real fix is band RMS rather than per-bin, a threshold in sigma rather than fixed %, persistence on the same band across frames, and a default band of <code>highpass_fc ... maxfreq</code> — i.e. it depends on R34's declared band. Decide then: rework on top of R34, or delete the hook and its GUI/headless/config surface outright.	TODO

#	Source	Requirement	Status
R40	Aug 2026	Sensor-fault detection. Disconnect a cable mid-run today and the app trends a near-zero reading as a valid healthy measurement — the machine appears to have improved. The textbook fix, reading the IEPE bias against a nominal 8–14 V window, is not available on this hardware: our coupler has a DC blocking capacitor on its output so the bias never reaches the scope on either coupling setting, and the 4000A ranges only to ± 20 V, so a 24 V supply would over-range even with a direct pre-cap tap. Software workaround (in progress): dead-channel detection from the AC signal — flag a channel whose band RMS falls below a per- ScopeSensor minimum-plausible floor, plus the open-circuit signature of a rail-ward step decaying at the coupler's high-pass time constant. Threshold to be set from measured separation between connected/still, connected/excited, shorted and open, not invented. Hardware route, if ever wanted: a coupler tap ahead of the blocking cap with a divider bringing 24 V inside ± 20 V, or a coupler exposing a fault output.	TODO

#	Source	Requirement	Status
R43	Aug 2026	Tachometer channel support — shaft speed is the denominator that turns a spectrum into a diagnosis: it names the lines (a 5.43x bearing tone moves 6.1 Hz between no load and full load on a 4-pole motor, 12 bins at 0.5 Hz), separates 2x line frequency from 2x running speed (80 CPM apart, different repairs), and gates trending so a load swing is not read as a condition change. Scope for the first cut: RPM display and trend, order cursors on the spectrum, an Order column in the peaks table, and speed-gated alarming via <code>valid_results()</code>. Deliberately out of scope: order-normalised resampling (a smeared spectrum is rejected via <code>SPEED_DRIFT_MAX_PCT</code>, not corrected — bearing analysis is done at steady state) and balancing phase reference (the shipped high-pass rotates 1x by 21° at 3600 RPM rising to 143° at 600, while amplitude stays correct, so nothing on screen would flag the error). Measured accuracy limit: full accuracy needs $\geq \sim 70$ samples per pulse (pulse rate $\leq \sim 600$ Hz) — 36000 RPM at 1 ppr, 600 RPM at 60 ppr, 35 RPM at 1024 ppr, so a high-line encoder is impractical and the UI must say so. Storage is per-frame edge times, not the tach waveform — $\sim 1400\times$ smaller and still enough to re-derive RPM at a different pulses/rev. Enables R6 and R24, and the envelope band trending left pending under R41.	Done — <code>rev80/tach.py</code>, channel roles, coherent simulation, collector wiring, HDF5 v5, speed gate, headless refusal, monitor v6, Tachometer tab with live preview, 1x marker and level. Closed out on a 4424A: 23 hardware tests pass, including an AWG sweep 300–10200 RPM within $\pm 0.2\%$, the AC-coupling failure reproduced electrically at 70% duty, and a clean 4-channel sustained run. Outstanding: shaft speed on the main display outside the dialog, and RPM on the trend plot (blocked on R33 — the trend already has two unit-based y-axes)
R44	Aug 2026	Tachometer support in <code>rev80-headless</code> — deferred, not rejected. Headless currently must <i>refuse</i> tach-role channels rather than ignore them: it builds its config from the same <code>devices/*.yaml</code> and iterates <code>enabled_channels</code>, so a GUI-configured tach channel would be high-passed, given an overall, trended and fed to the anomaly hooks as though it were vibration. Measured on a 5 % duty pulse train at 1800 RPM through the real <code>process_sample</code>: overall 1514.9 mV, crest 5.00, kurtosis 15.94, 63 spectral peaks — an analyst reviewing that session concludes a bearing is failing badly. It also drifts on nothing: a tach LED ageing from 5.0 V to 4.5 V moves that channel's overall by exactly $\sim 10\%$, the shipped <code>RmsThresholdHook</code> threshold, on three consecutive frames. Revisit once R43 lands.	TODO

#	Source	Requirement	Status
R45	Sep 2026	Motor-state judgement from the tachometer — a stopped shaft and a disconnected cable can produce an identical block: flat and near zero, whenever the reflector happens to sit away from the sensor. Only the parked-reflector case (flat at the high rail) and turning-below-the-floor (real pulses, fewer than MIN_EDGES) are distinguishable today, so <code>no_signal</code> is reported as exactly that and nothing is inferred about the machine. Making "motor stopped" trustworthy needs <i>history</i> — a channel that read cleanly and then ceased is a stop; one that never read is a setup problem — which is a small state machine belonging with whatever consumes the state rather than in <code>tach.py</code>. Until then the GUI must not claim a machine is stopped on the strength of a flat line.	TODO
R46	Sep 2026	Surface velocity from reflector size and duty cycle — with reflector arc length L, measured duty d and shaft rate f, the reflector subtends d of a revolution, so circumference is L/d and surface velocity is $v = f \cdot L/d$. The tape doubles as a shaft-diameter measurement, giving surface speed without anyone measuring the shaft — directly useful on rollers, belts and web handling. Duty cycle is now captured (<code>TachResult.duty_cycle / .pulse_widths_s</code>, persisted in v5), so the remaining work is the reflector-size input and the $v = f \cdot L/\text{duty}$ readout. Sensor geometry — an optical tach's spot width, a proximity probe's inductive field — inflates the observed duty; by decision the operator accounts for that when measuring the tape or key, so no correction is applied in software.	TODO

#	Source	Requirement	Status
R47	Sep 2026	Settling indicator while the measurement chain stabilises. The first frame of a stream reads conspicuously high — approximately 2x on the overall — and is displayed, trended and alarmed on as though it were a valid measurement. Real analysers show “acquiring”/“settling” and withhold the reading until the chain has stabilised. Three candidates to separate before fixing, and they are not equally likely: the IEPE coupler’s own AC-coupling RC transient after <code>ps4000aRunStreaming</code> (real electrical signal, which no digital filter can remove, and the most probable of the three); the <code>resample_poly</code> anti-alias FIR’s block-edge transient; and the first-block high-pass state — noting <code>filter_block</code> already seeds <code>zi</code> from the block mean via <code>_seed_zi</code> rather than from zero, so that one should already be handled and is the least likely. Measure the settling time rather than assuming it, then withhold or flag frames inside it and exclude them from the trend, the baseline and anomaly evaluation exactly the way overflow/degraded frames already are (<code>valid_results()</code>). Raised while profiling GUI responsiveness (Sep 2026); deliberately deferred there as out of scope.	TODO
—	Future	Proximity probe support	Done — scope sensor with EU in displacement units
—	Future	Web portal for data sharing	Abandoned
—	Future	MCC DAQ tooling (USB-1608FS-Plus)	Abandoned — PicoScope oscilloscope replaces DAQ for all current use cases

Architecture Evolution Summary

Era	Hardware	Key modules	Format
Proto (Feb–Aug 2025)	Digiducer USB audio	monolithic	—
Phase 0 (Nov–Dec 2025)	Digiducer	sensor, sample, simulation, digiducer	pickle
Phase 1b (Jan 2026)	Digiducer	+ HDF5, refactored AcquisitionSettings	HDF5

Era	Hardware	Key modules	Format
Phase 2 (Mar 2026)	PicoScope 4000A	+ picoscope, scope_sensor, scope_sensor_registry	HDF5 multi-channel
Phase 2a (Mar 2026)	PicoScope	+ ChannelResult, multi-channel pipeline, trend	HDF5 multi-channel
Phase 2b (Mar 2026)	PicoScope + AWG	+ signal generator, Welch controls, derived AcquisitionSettings	HDF5 + YAML config
Windows packaging (Mar–May 2026)	PicoScope	+ _paths.py , _pico_loader.py , build/collect_pico_dlls.py ; PyInstaller + Inno Setup pipeline	HDF5 v4 + frozen one-dir bundle
Phase 4 (May 2026)	PicoScope	mV-domain VibeSample, (M,5) trend, process_samples()	HDF5 v4
Monitor Mode (May–Jun 2026)	PicoScope	+ monitor/ (gate , controller , writer , anomaly , session), headless.py ; config split into acquisition.yaml + devices/	HDF5 v4 + one session.h5 v5 per session
Rebrand → Rev80 (Aug 2026)	PicoScope	src-layout move to src/rev80/ ; doc/ / scripts/ / build/ reorg	HDF5 v4; data ~/Documents/Rev80/ , config ~/.config/rev80/
Anti-alias (Aug 2026)	PicoScope 4424A	picoscope.antialias_decimate() — mandatory oversample → Kaiser FIR → decimate; streaming-rate degradation watchdog; user-facing lowpass removed	HDF5 v4 + per-frame degraded flag
Audit round 1 — measurement validity (Aug 2026)	PicoScope	+ _dsp.py ; persistent per-channel high-pass state, Hann/Tukey-tapered integration, nperseg = blocksize , true (float) sample rate	HDF5 v4, overflow/ degraded read back
Audit round 2 — integrity & diagnostics (Aug 2026)	PicoScope	+ envelope.py , peaks.py ; declared measurement band, crest factor / kurtosis, GenerateBearingVibration() oracle	HDF5 v4 + band, crest/ kurtosis trends, peak_threshold_db
Spectral averaging (Aug 2026)	PicoScope	_psd_and_overalls_for() ; power-domain averaging over N frames, above the per-frame PSD cache	HDF5 v4 unchanged — the average is a view on stored frames
CLI & Linux packaging (Aug 2026)	PicoScope	+ desktop.py (XDG launcher entry + hicolor icons); unified rev80 CLI with a headless subcommand; CONTRIBUTING.md split out; resource_path() fixed for non-editable installs	HDF5 v4; data_dir() always ~/Documents/Rev80/data , in dev and frozen builds alike

Era	Hardware	Key modules	Format
Raw-stream retention (Aug–Sep 2026)	PicoScope	fixed <code>RAW_SAMPLERATE_HZ</code> acquisition rate vs. derived display rate (exactly $2.56 \times \text{maxfreq}$ from Sep); <code>decimate_to_rate()</code> , <code>eu_scaled_raw()</code> , <code>current_frame()</code> , <code>simulation._RawRateView</code> , <code>scripts/validate-streaming-capacity</code>	HDF5 — every stored frame is the raw-rate capture, independent of <code>maxfreq</code>
Tachometer (Sep 2026)	PicoScope + optical tach	+ <code>tach.py</code> , channel roles, speed gate, Tachometer tab, 1× marker, <code>GenerateTachPulse /</code> <code>GenerateMachineWithTach</code>	HDF5 v5 — edge times, no <code>data</code> on tach channels; <code>session.h5 v6</code> (<code>rpm / speed_ok</code>)